

دليل مذاكرة شامل · بالعربي + المصطلحات بالإنجليزي

# شهادة AWS Solutions Architect

## الدليل الكامل للمفاهيم

**AWS Certified Solutions Architect – Associate · SAA-C03**

شرح مبسّط ومركّز لكل مفهوم في ال checklist بتاعك، مقسوم على الأربع domains الرسمية، مع جداول مقارنة للنقاط اللي بيتلغبط فيها الناس، ورسوم توضيحية لل architectures المهمة، وصناديق «امتى تستخدمه» و«نقطة امتحان» اللي بتفرق معاك في القاعة.

**4**

Domains

**3**

سنوات صلاحية

**720**/1000

درجة النجاح

**130**

دقيقة

**65**

سؤال

# محتويات الدليل

## 1 نظرة عامة وخريطة المذاكرة

أوزان الـ domains + إزاي تستخدم الدليل

## 2 Architectures – Domain 1 آمنة

IAM · Encryption · Network Security · Threat Detection · Federation

## 3 Architectures – Domain 2 مرنة

Multi-AZ/Region · ELB · Auto Scaling · DB HA · Storage · DNS · Decoupling

## 4 Architectures – Domain 3 تصميم Architectures عالية الأداء

Compute · Storage · DB · Analytics · Networking · API

## 5 Architectures – Domain 4 تصميم Architectures موفّرة للتكلفة

EC2 Pricing · S3 Classes · Serverless · أدوات التكلفة

## 6 Architectural Patterns + Well-Architected

الـ patterns الجاهزة + الـ 6 pillars

## 7 خطة مذاكرة 8 أسابيع + مصادر

جدول أسبوعي + أفضل المصادر

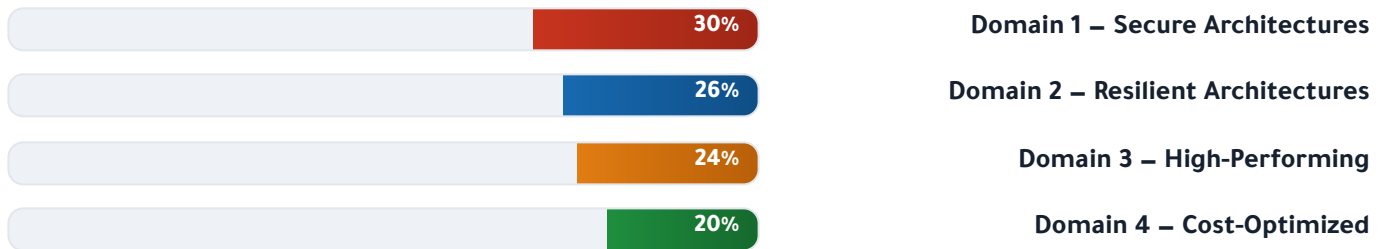
# نظرة عامة وخريطة المذاكرة

i

امتحان الـ SAA-C03 مش بيسألك «إيه تعريف الـ service ده؟» – بيسألك سؤال سيناريو (شركة عندها كذا، عايزة كذا، إيه أنسب حل؟) ويديك 4 اختيارات كلها «بتشتغل»، بس واحد بس هو **الأنسب** من حيث الأمان/المرونة/الأداء/التكلفة. عشان كده التركيز في الدليل ده على «**امتى تستخدم إيه**» وعلى الفروقات بين الخدمات المتشابهة، مش على الحفظ المجرّد.

## أوزان الـ Domains في الامتحان

وزّع وقت مذاكرتك بنفس النسب دي تقريبًا – الأمان والمرونة لوحدهم أكثر من نص الامتحان:



## إزاي تقرأ الدليل ده

### صندوق «خلي بالك»

فخ شائع أو معلومة بيغلط فيها أغلب الناس – احفظها كويس.

### جداول المقارنة

كل بند فيه «X vs Y» في الـ checklist بقى جدول صريح – دي أعلى نسبة عائد للمذاكرة.

### صندوق «امتى تستخدمه»

السطر اللي بيفرق في سؤال السيناريو – الكلمة المفتاحية اللي لو شُفّتها في السؤال تروح على الخدمة دي على طول.

### صندوق «نقطة امتحان»

رقم أو حقيقة بيحبوا يلعبوا بيها في الـ distractors (زي مدة الـ Spot notice أو حدود الـ Lambda).

### قاعدة ذهبية للامتحان

لو السؤال قال «**most cost-effective**» فكّر Serverless / Spot / S3 tiers. لو قال «**highly available**» أو «**fault tolerant**» فكّر Multi-AZ. لو قال «**disaster recovery / another region**» فكّر Multi-Region. لو قال «**decouple**» فكّر SQS/SNS. لو قال «**without managing servers**» فكّر Lambda/Fargate.

30%

Secure

# تصميم Architectures آمنة

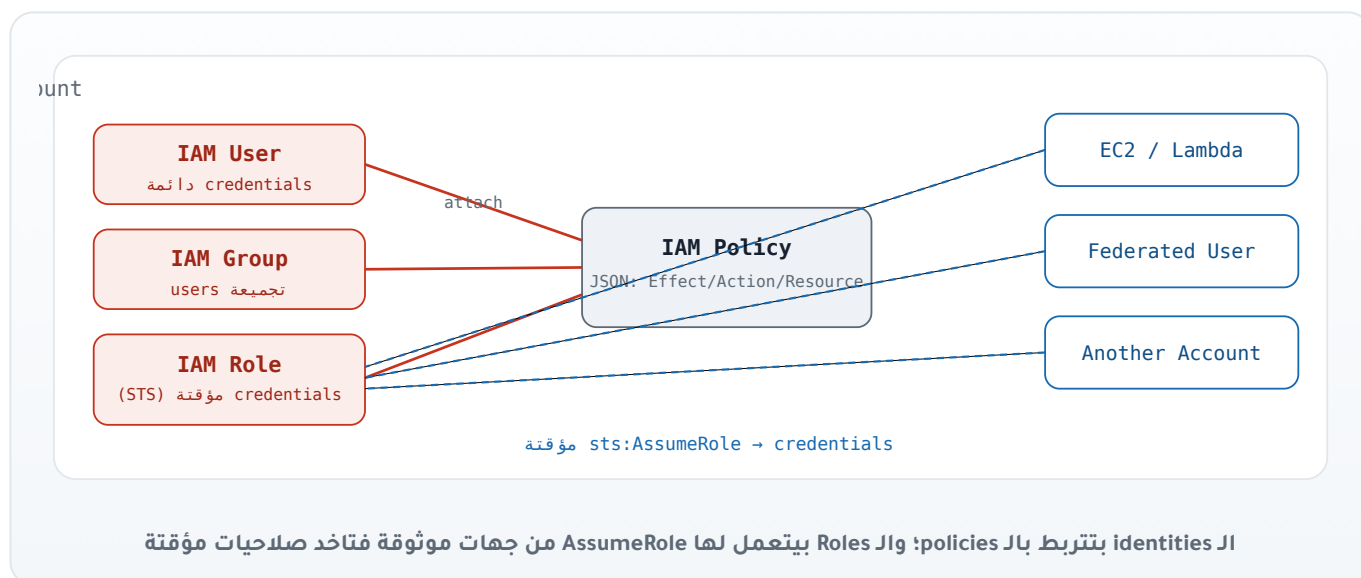
1

أكبر domain في الامتحان. الفكرة الحاكمة هنا: **Least Privilege** (أقل صلاحية ممكنة)، و **Defense in Depth** (طبقات حماية متعددة)، و **تشفير كل حاجة** at-rest و in-transit.

## IAM وإدارة الصلاحيات

### المكونات الأساسية: Users · Groups · Roles · Policies

ال **User** هو هوية لشخص أو تطبيق ولها credentials دائمة. ال **Group** تجميعية users بتأخذ نفس ال policies (للتنظيم بس – مش بتنفع تتعمل لها login). ال **Role** هوية مؤقتة بتتعمل لها **assume** وبتدي credentials مؤقتة عن طريق **STS** – دي الطريقة الصح لـ EC2, Lambda, و cross-account. بدل ما تحط access keys في الكود. ال **Policy** دوكيومنت **JSON** بيحدد إيه مسموح/ممنوع.



### أنواع ال Policies: Managed vs Inline vs Permission Boundaries

## الفرق بين أنواع السياسات

| النوع               | إيه هو                                                              | امتى تستخدمه                                                     |
|---------------------|---------------------------------------------------------------------|------------------------------------------------------------------|
| AWS Managed         | جاهزة من AWS (زي <code>AdministratorAccess</code> ) ويتحدث تلقائيًا | بداية سريعة / use-cases شائعة                                    |
| Customer Managed    | إنت بتعملها وتعيد استخدامها على كذا identity                        | الأفضل عمومًا: قابلة لإعادة الاستخدام + versioning               |
| Inline              | متلصوقة بـ identity واحدة بس (علاقة 1:1)، بتسمح مع الـ identity     | لما تحتاج علاقة حصرية صارمة بين policy و identity                |
| Permission Boundary | سقف أقصى للصلاحيات – مش بيدّي صلاحية، بيحدّد الحد الأعلى            | تحدّد أقصى ما يقدر يعمل user/role حتى لو اتلصقت له policies أكبر |

### إزاي بتحسب الصلاحية الفعلية

الصلاحية النهائية = تقاطع (Identity policy) ∩ (Permission Boundary) ∩ (SCP)، وأي **Explicit Deny** في أي طبقة يغلب على أي Allow. الـ Deny دايماً بيكسب.

## Cross-Account والوصول AssumeRole

بدل ما تعمل users في كل حساب، بتعمل **Role** في الحساب صاحب المورد، وتديها **trust policy** بتسمح للحساب الثاني يعمل لها `sts:AssumeRole`. ده الأسلوب الموصى به للوصول بين الحسابات.

### IAM Conditions

`aws:SourceIp` · `aws:RequestedRegion`

بتضيف شروط على الـ policy: زي تسمح بالوصول من نطاق IP معين (`aws:SourceIp`)، أو region محدد (`aws:RequestedRegion`)، أو تفرض MFA (`aws:MultiFactorAuthPresent`)، أو وقت معين. أداة قوية للـ least privilege.

## Service Control Policies (SCPs) و AWS Organizations

الـ **Organizations** بتجمع كذا حساب تحت مظلة واحدة (consolidated billing + إدارة مركزية). الـ **SCP** بتتطبق على مستوى الـ OU/الحساب وتحدّد سقف الصلاحيات لكل الـ identities في الحساب – حتى الـ **root** بتاع الحساب العضو بيتقيّد بيها.

### ⚠ خلي بالك

SCP **مش بتدي** صلاحيات – بس بتسمح/بتمنع كحد أقصى. لازم كمان IAM policy تدي الصلاحية فعليًا. وكمان مش بتأثر على الـ management account نفسه.

## IAM Identity Center (الاسم الجديد لـ AWS SSO)

نقطة دخول مركزية واحدة (Single Sign-On) للوصول لكل حسابات الـ Organization ولتطبيقات SaaS. مع إمكانية الربط بـ Active Directory أو IdP خارجي. أنسب لإدارة وصول الموظفين على كذا حساب.

## Access Analyzer · MFA · مستويات الصلاحية

- **MFA**: طبقة ثانية للتوثيق (Virtual MFA زي Google Authenticator، أو hardware key). دايماً فَعْلَهَا على ال root وامسح ال access keys بتاعته.
- **IAM Access Analyzer**: بيحلل ال policies ويكشف أي مورد متشارك مع جهة خارجية (external access) – أداة مراجعة أمنية.
- **Power User vs Admin vs Read-Only**: ال Admin كل حاجة: ال Power User كل حاجة ماعدا/ إدارة ال IAM والحسابات: ال Read-Only يشوف بس.

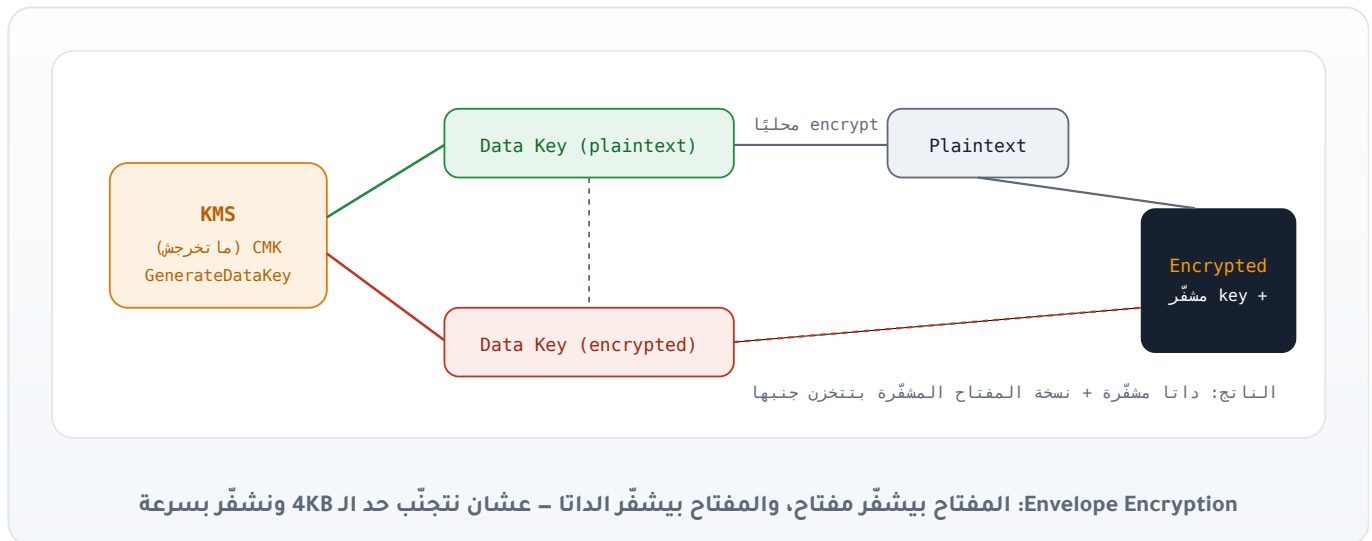
## التشفير والأسرار 🔒

### AWS KMS CMK • Data Keys • Key Policies

خدمة مركزية لإدارة مفاتيح التشفير. ال **CMK / KMS Key** هو المفتاح الأساسي (مش بيخرج من KMS أبداً). ممكن يكون **AWS-managed** أو **Customer-managed** (تتحكم في ال rotation وال key policy). ال **Key Policy** هي اللي بتحدد مين يقدر يستخدم/يدير المفتاح.

### Envelope Encryption

بدل ما تبعت الداتا كلها ل KMS (فيه حد أقصى 4KB)، KMS بيطلع لك **Data Key**: نسخة plaintext تشقر بيها الداتا محلياً، ونسخة **مشفرة** تخزنها جنب الداتا. وقت فك التشفير، KMS بيفك ال data key المشقر، وبيه تفك الداتا. ده اللي بيخلي تشفير ملفات كبيرة ممكن وسريع.



### KMS vs CloudHSM – امتى تستخدم كل واحد

| CloudHSM                                                                | AWS KMS                                       |         |
|-------------------------------------------------------------------------|-----------------------------------------------|---------|
| single-tenant. جهاز HSM مخصص ليك                                        | multi-tenant مُدار بالكامل                    | النوع   |
| إنت اللي بتتحكم بالكامل في المفاتيح                                     | AWS بتدير ال hardware                         | التحكم  |
| FIPS 140-2 Level 3 مخصص                                                 | FIPS 140-2 Level 3                            | المعيار |
| متطلب compliance صارم يفرض single-tenant أو تحكّم كامل/ مفاتيح SSL خاصة | الافتراضي لمعظم الحالات + تكامل مع كل الخدمات | امتى    |

## S3 Server-Side Encryption: SSE-S3 · SSE-KMS · SSE-C

### أنواع تشفير S3 من جهة السيرفر

| النوع   | مين يملك المفتاح                | المميزات / امتي                                                                                    |
|---------|---------------------------------|----------------------------------------------------------------------------------------------------|
| SSE-S3  | AWS بالكامل (AES-256)           | أبسط حاجة – تشفير افتراضي بدون مجهود (دلو قتي مفعل تلقائيًا على كل buckets)                        |
| SSE-KMS | مفتاح KMS بتاعك                 | تحكم + audit trail في CloudTrail لكل استخدام للمفتاح + rotation. (انتبه لحد الـ KMS request quota) |
| SSE-C   | إنت بتوفر المفتاح في كل request | عايز تتحكم في المفتاح بنفسك بره AWS بالكامل (لازم HTTPS)                                           |

## Secrets Manager vs Parameter Store

### إدارة الأسرار والإعدادات

| SSM Parameter Store                         | Secrets Manager                   |                 |
|---------------------------------------------|-----------------------------------|-----------------|
| إعدادات config + أسرار بسيطة (SecureString) | أسرار (DB passwords, API keys)    | الغرض           |
| لأ (إلا لو عملته يدوي بـ Lambda)            | نعم (مدمج، ول RDS بضغطة)          | Rotation تلقائي |
| Standard مجاني · Advanced مدفوع             | مدفوعة لكل secret                 | التكلفة         |
| config values كثير + ميزانية محدودة         | محتاج rotation تلقائي أو تكامل DB | امتي            |

## AWS Certificate Manager (ACM) – Public vs Private CAs

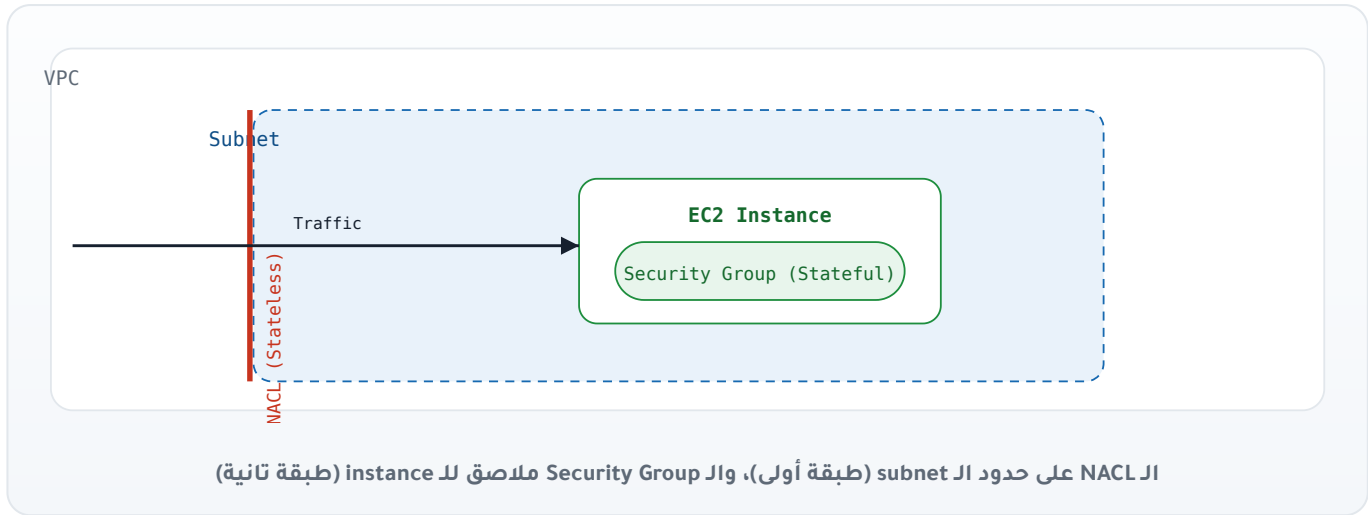
بيديك وبيدير شهادات SSL/TLS مجانًا. الـ **Public certs** للمواقع المتاحة على الإنترنت (تتكامل مع ALB/CloudFront/API Gateway مع تجديد تلقائي). الـ **Private CA** لإصدار شهادات داخلية لشبكتك الخاصة.

### ⚠️ فخ شائع

شهادات ACM العامة بتتكامل مع ALB و CloudFront و API Gateway بس – **مش** بتنفع تتركب على EC2 مباشرة. وكماني شهادة CloudFront لازم تكون في region `us-east-1`.

## 🛡️ أمان الشبكة

### Security Groups (Stateful) vs NACLs (Stateless)



### أهم فرق بيتسأل عنه كثير

| NACL                                                          | Security Group                        |           |
|---------------------------------------------------------------|---------------------------------------|-----------|
| على ال subnet                                                 | على ال instance / ENI                 | المستوى   |
| <b>Stateless</b> – لازم تسمح لل inbound و outbound كل على حدة | <b>Stateful</b> – الرد يُسمح تلقائيًا | Stateful؟ |
| Allow و Deny (مرتبة برقم)                                     | Allow بس (مفيش Deny)                  | القواعد   |
| بالترتيب من الأصغر رقمًا، أول match يوقف                      | كل القواعد مع بعض                     | التقييم   |

### VPC Flow Logs

بتسجل معلومات عن ال IP traffic الداخل والخارج من ال VPC/subnets/ENIs (مين، فين، اتقبل ولا اترفض). بتروح ل CloudWatch Logs أو S3. أداتك الأولى لتشخيص مشاكل الاتصال والتحقيق الأمني. **مش** بتشوف محتوى ال packets.

### WAF · Shield · Firewall Manager · Network Firewall

#### طبقات الحماية على مستوى الشبكة والتطبيق

| الخدمة           | الطبقة         | بتحمي من إيه                                                                                |
|------------------|----------------|---------------------------------------------------------------------------------------------|
| WAF              | Layer 7 (HTTP) | SQL injection · XSS · rate-based rules · bots<br>ALB/CloudFront/API – بتتركب على GW/AppSync |
| Shield Standard  | L3/L4          | <b>مجاني وتلقائي</b> ضد هجمات DDoS الشائعة                                                  |
| Shield Advanced  | L3/L4/L7       | حماية DDoS متقدمة + فريق DRT + تعويض مالي عن تكاليف التوسع وقت الهجوم (مدفوع)               |
| Firewall Manager | إدارة مركزية   | تطبيق قواعد WAF/Shield/SG عبر <b>كل حسابات</b> ال Organization                              |
| Network Firewall | L3-L7          | جدار ناري مُدار على مستوى ال VPC (IPS/IDS، فلترة domains)                                   |

### VPC Endpoints: Gateway vs Interface · PrivateLink

بتخلي مواردك في ال VPC توصل لخدمات AWS من غير ما تطلع على الإنترنت (أمن وأرخص في ال data transfer).

## Gateway Endpoint vs Interface Endpoint

| Interface Endpoint (PrivateLink)       | Gateway Endpoint        |         |
|----------------------------------------|-------------------------|---------|
| أغلب خدمات AWS + خدمات partners الخاصة | S3 و DynamoDB بس        | الخدمات |
| subnet ب private IP جوه ال ENI         | route table في ال entry | الآلية  |
| بالساعة + بال GB                       | مجاني                   | التكلفة |

### نقطة امتحان 💡

لو السؤال قال «نوصل ل S3 من غير internet gateway» والاختيارات فيها endpoint → الإجابة **Gateway Endpoint** (لأنه مجاني ومخصص ل S3). ال **PrivateLink** هو الاسم التسويقي لل Interface Endpoint.

## كشف التهديدات وال Compliance 🕵️

كل خدمة بتعمل إيه بالضبط – الكلمة المفتاحية تفرق في السؤال

| الخدمة       | بتعمل إيه                                                    | الكلمة المفتاحية                            |
|--------------|--------------------------------------------------------------|---------------------------------------------|
| GuardDuty    | كشف تهديدات ذكي بتحليل CloudTrail + VPC Flow Logs + DNS logs | «intelligent threat detection» / نشاط مشبوه |
| Inspector    | فحص ثغرات (vulnerabilities) في EC2 · ECR · Lambda            | «vulnerability / CVE scanning»              |
| Macie        | اكتشاف بيانات حساسة (PII) جوه S3 باستخدام ML                 | «sensitive data in S3»                      |
| Security Hub | لوحة موحدة بتجمع نتائج كل الخدمات الأمنية                    | «aggregate / central security view»         |
| Detective    | تحليل السبب الجذري للحوادث (root cause) ورسم العلاقات        | «investigate / root cause»                  |
| Config       | تتبع تغييرات الإعدادات + قواعد compliance + drift            | «configuration compliance / drift»          |
| CloudTrail   | سجل كل API calls في الحساب (مين عمل إيه وإمتى)               | «who did what – API audit»                  |

الثلاثي اللي بيتلغبط: CloudTrail vs CloudWatch vs Config

| AWS Config                | CloudWatch                 | CloudTrail              |            |
|---------------------------|----------------------------|-------------------------|------------|
| «إعدادات المورد متوافقة؟» | «المورد شغال كويس؟ (أداء)» | «مين عمل ال action ده؟» | بيجواب على |
| Config history + rules    | Metrics · Logs · Alarms    | API activity / audit    | البيانات   |
| أي EBS مش مشقّر           | alarm → CPU عدّي 80%       | مين مسح ال bucket       | مثال       |

## Identity Federation 🔑

Cognito: User Pools vs Identity Pools

## أكثر نقطة بتتلفط في Cognito

| Identity Pools (Federated Identities)                           | User Pools                                                                |                 |
|-----------------------------------------------------------------|---------------------------------------------------------------------------|-----------------|
| <b>Authorization</b> – بتدي credentials مؤقتة للوصول لخدمات AWS | <b>Authentication</b> – مين إنت؟ (sign-up/sign-in + directory) للمستخدمين | <b>الوظيفة</b>  |
| AWS credentials مؤقتة (عبر STS)                                 | JWT tokens                                                                | <b>بيدي إيه</b> |
| تخلي المستخدم يرفع مباشرة على S3 بصلاحيه مؤقتة                  | تسجيل دخول مستخدم تطبيق الموبايل                                          | <b>مثال</b>     |

**SAML 2.0** للربط مع IdP مؤسسي (زي Active Directory) للموظفين. **Web Identity Federation** و **OIDC** للربط مع مزودين زي Google/Facebook/Amazon – غالبًا بيتعملوا جوه Cognito Identity Pools لتطبيقات الويب والموبايل.

### 💡 خلاصة 1 Domain في سطر

Roles بدل keys · شق كل حاجة بـ (stateful) SG · KMS لـ instance و (stateless) NACL لـ GuardDuty · subnet للتهديدات و Macie للبيانات الحساسة · CloudTrail = مين عمل إيه.

26%

Resilient

## تصميم Architectures مرنة

2

الهدف: النظام يفضل شغّل حتى لو حاجة وقعت. فرّق بين **High Availability** (تتعافى بسرعة من عطل مكوّن) و **Faulty Tolerance** (تستحمل العطل من غير ما حد يحس). القاعدة الذهبية: **وزّع على كذا AZ**، وللكوارث الكبيرة وزّع على كذا Region.

Multi-AZ · Multi-Region · DR 

## RTO vs RPO – لازم تحفظهم

## أهم تعريفين في ال Disaster Recovery

| RPO – Recovery Point Objective                       | RTO – Recovery Time Objective                 |          |
|------------------------------------------------------|-----------------------------------------------|----------|
| قد إيه <b>الداتا</b> اللي تستحمل تخسرها (آخر backup) | قد إيه <b>الوقت</b> اللي تستحمل تكون فيه down | بيقيس    |
| «نرجع لأي نقطة زمنية؟»                               | «إمتى نرجع نشغل؟»                             | السؤال   |
| backups أكثر تكرارًا / replication                   | أتمتة ال standby / failover جاهز              | تقلله بـ |

## استراتيجيات ال DR الأربعة

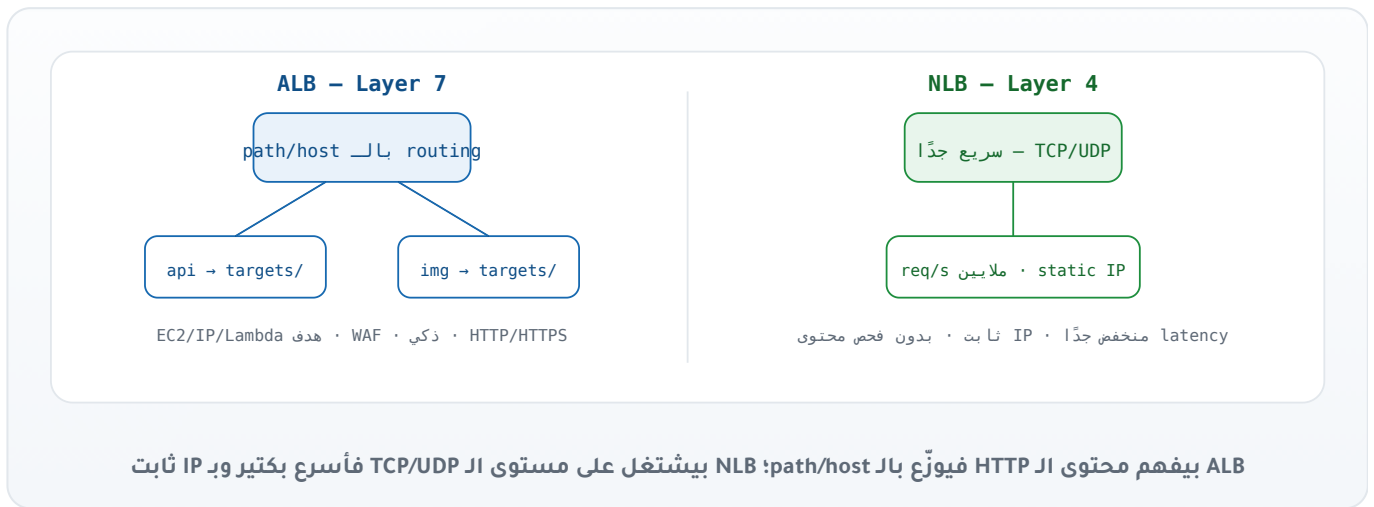


## مقارنة سريعة بال RTO/RPO والتكلفة

| الفكرة                                     | التكلفة | RTO/RPO       | الاستراتيجية             |
|--------------------------------------------|---------|---------------|--------------------------|
| backups فقط، تبني البنية وقت الكارثة       | الأرخص  | ساعات         | Backup & Restore         |
| ال DB شغّل، باقي المكونات مطفية وجاهزة     | منخفضة  | عشرات الدقائق | Pilot Light              |
| بيئة مصغرة شغالة دايمًا، تكبرها عند الحاجة | متوسطة  | دقائق         | Warm Standby             |
| نسختان كاملتان تخدمان الترافيك معًا        | الأعلى  | ≈ صفر         | Multi-Site Active/Active |

**AWS Backup**: إدارة مركزية للـ backups عبر خدمات كثير (EBS, RDS, DynamoDB, EFS...) بـ policies وجدولة موحدة وعبر الحسابات والـ regions.

## Load Balancing



### أنواع الـ Load Balancers

| النوع | الطبقة       | امتى تستخدمه                                                      |
|-------|--------------|-------------------------------------------------------------------|
| ALB   | L7 (HTTP/S)  | تطبيقات ويب · routing بالـ microservices · path/host · هدف Lambda |
| NLB   | L4 (TCP/UDP) | أداء عالي / latency منخفض جدًا · IP ثابت · ملايين الاتصالات       |
| GWLB  | L3 (Gateway) | تمرير الترافيك على أجهزة أمان طرف ثالث (firewalls/IDS)            |
| CLB   | L4/L7 قديم   | Legacy – ماتختار هوش في تصميم جديد                                |

- **Target Groups**: ALB بيوزع على EC2 أو IP أو Lambda.
- **Sticky Sessions**: تربط مستخدم بنفس الـ target (مفيدة للـ stateful apps).
- **Connection Draining** (Deregistration Delay): يخلص الـ requests الجارية قبل ما يشيل الـ target.

## Auto Scaling

الـ **ASG** يحافظ على عدد الـ instances بين Min و Max و Desired، ويستبدل أي instance بيوقع (self-healing)، ويوزعهم على كذا AZ. بيبنى من **Launch Template** (الأحدث والموصى به – بيدعم versioning) بدل الـ Launch Configuration القديم.

## سياسات التوسّع (Scaling Policies)

| السياسة         | الفكرة                                   | امتى                                      |
|-----------------|------------------------------------------|-------------------------------------------|
| Target Tracking | تحافظ على metric عند رقم (مثلاً 50% CPU) | <b>الأبسط والأشهر</b> – اضبط الهدف وسيبته |
| Step Scaling    | تزداد/تقلل خطوات حسب شدة التجاوز         | تحكم أدق في حجم الاستجابة                 |
| Scheduled       | توسع في وقت معروف مسبقاً                 | جمل متوقع (مثلاً 9 صباحاً كل يوم)         |
| Predictive      | ML يتنبأ بالجمل ويوسع قبلها              | أنماط دورية متكررة                        |

**Cooldown / Warm-up:** فترة انتظار بعد كل scaling عشان الـ metrics تستقر قبل قرار جديد. **Application Auto Scaling:** نفس الفكرة بس لخدمات ثانية زي ECS و DynamoDB و Aurora replicas.

## قواعد البيانات (HA & DR)



### RDS Multi-AZ vs Read Replicas – أكثر نقطة بتلغبط

| Multi-AZ                           | Read Replicas                  |              |
|------------------------------------|--------------------------------|--------------|
| توافرية / تعافٍ من الأعطال (HA/DR) | توسيع القراءة (scalability)    | الغرض        |
| متزامن (synchronous)               | غير متزامن (asynchronous)      | Replication  |
| <b>لا</b> – الـ standby خامل       | <b>نعم</b>                     | بيخدم قراءة؟ |
| <b>تلقائي</b>                      | يمكن ترقيته يدوياً لـ primary  | Failover     |
| (عادةً نفس الـ region)             | <b>نعم</b> – يصلح لـ DR إقليمي | Cross-Region |

### Amazon Aurora وعائلتها

- Aurora:** متوافق مع MySQL/PostgreSQL، تخزين بيتنسخ 6 نسخ عبر 3 AZs تلقائياً، أداء أعلى من RDS العادي.
- Aurora Serverless v2:** بيوسع السعة تلقائياً up/down حسب الجمل – مثالي للأحمال المتقطعة/غير المتوقعة.

- **Aurora Global Database**: نسخ عبر regions بـ  $RPO \approx 1$  ثانية و  $RTO < 1$  دقيقة – أفضل خيار لـ DR إقليمي للقواعد العلائقية.

#### 💡 نقطة امتحان

«قاعدة علائقية + DR عبر RTO/RPO regions منخفض جدًا» → **Aurora Global Database**. «جمل غير متوقع/متقطع + معاييرش أدير سعة» → **Aurora Serverless v2**.

#### DynamoDB · ElastiCache

- **DynamoDB Global Tables**: multi-region **active-active** (تكتب وتقرأ في أي region) – للتطبيقات العالمية.
- **DynamoDB PITR**: استرجاع لأي لحظة خلال آخر 35 يوم (حماية من الحذف الغلط).
- **ElastiCache Redis**: يدعم cache → Cluster Mode + replication + Multi-AZ عالي التوافر.

#### 📦 مرونة التخزين

##### تكرار/حماية بيانات التخزين

| الميزة                            | إيه هي                                                                                                                   |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>S3 Versioning + MFA Delete</b> | تحتفظ بكل نسخ الأوبجكت؛ MFA Delete يمنع الحذف النهائي إلا بـ MFA                                                         |
| <b>CRR vs SRR</b>                 | <b>Cross-Region Replication</b> لـ DR/الامتثال/تقليل latency عالميًا: <b>Same-Region</b> لـ log aggregation/فصل البيانات |
| <b>S3 Object Lock (WORM)</b>      | <b>Governance</b> : يقدر يكسره صاحب صلاحية خاصة · <b>Compliance</b> : <b>محدث</b> يقدر يحذف/يعدل حتى الـ root خلال المدة |
| <b>EBS Snapshots</b>              | نسخ تزايدية على S3، تنسخها عبر regions لـ DR                                                                             |
| <b>EFS</b>                        | تخزين ملفات إقليمي موزّع تلقائيًا عبر AZs (يتشارك بين instances كتير)                                                    |

#### 🌐 DNS والتوجيه العالمي

## سياسات توجيه Route 53 – كل واحدة وامتي

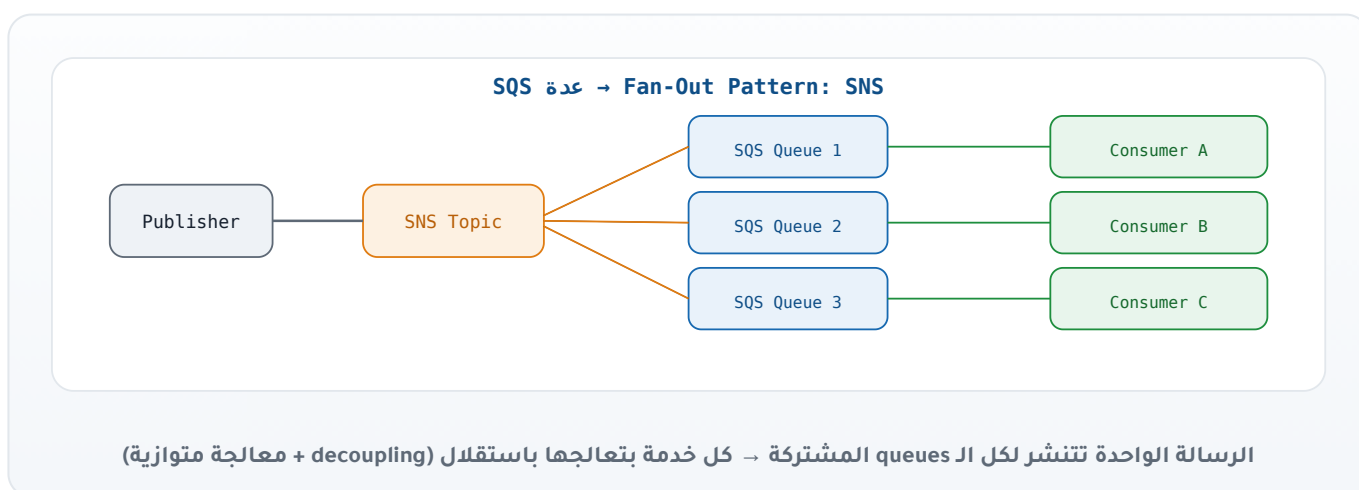
| السياسة           | بتعمل إيه                                                   |
|-------------------|-------------------------------------------------------------|
| Simple            | سجل واحد، بدون منطق                                         |
| Weighted          | توزيع بنسب (مثلاً 90/10) – مفيد للـ A/B testing والـ canary |
| Latency-based     | يوجّه للـ region الأقل latency للمستخدم                     |
| Failover          | primary/secondary مع health check – يحوّل للاحتياطي لو وقع  |
| Geolocation       | حسب موقع المستخدم (للامثال/المحتوى المحلي/اللغة)            |
| Geoproximity      | حسب المسافة الجغرافية مع إمكانية «bias» لتكبير/تصغير منطقة  |
| Multivalue Answer | يرجّع كذا IP صحي (load balancing بسيط على مستوى DNS)        |

## Global Accelerator vs CloudFront – بيتلغبطوا كتير

| Global Accelerator                                                   | CloudFront                           |           |
|----------------------------------------------------------------------|--------------------------------------|-----------|
| بيحسن الـ routing على شبكة AWS (مثل بيـ cache)                       | CDN – بيـ cache المحتوى عند الـ edge | النوع     |
| تطبيقات TCP/UDP غير قابلة للـ IP + cache ثابت + failover إقليمي سريع | محتوى ثابت/فيديو/مواقع (HTTP)        | الأنسب لـ |
| static anycast IP 2                                                  | أسماء CloudFront                     | IP        |

**CloudFront Origins:** ممكن S3 أو ALB أو EC2 أو أي custom origin، مع **Origin Failover** (origin أساسي واحتياطي) لمزيد من المتانة.

## Decoupling والرسائل



## SNS و SQS Standard vs FIFO

| SNS                            | SQS FIFO                 | SQS Standard                       |           |
|--------------------------------|--------------------------|------------------------------------|-----------|
| pub/sub (push) – لكل المشتركين | طابور مرتّب              | طابور (pull) – مستهلك واحد للرسالة | النمط     |
| – (FIFO topic متاح)            | مضمون + exactly-once     | best-effort (غير مضمون)            | الترتيب   |
| عالية                          | محدودة (مئات/آلاف/ثانية) | غير محدودة تقريبًا                 | الإنتاجية |

- **Visibility Timeout**: مدة إخفاء الرسالة بعد استلامها عشان متتالجتش مرتين.
- **DLQ**: طابور للرسائل الي فشلت معالجتها بعد عدد محاولات (لتحليلها لاحقًا).
- **Delay Queue**: تأخير ظهور الرسالة لفترة محددة.
- **EventBridge**: event bus يوصل أحداث من خدمات AWS و SaaS لأهداف كثير بقواعد + Scheduler.
- **Step Functions**: تنسيق **Standard** workflows (state machine) لطويلة المدى/ **Express**, audit, لعالية التردد وقصيرة.

### 💡 خلاصة 2 Domain في سطر

وزّع على ALB · AZs للويب و NLB للسرعة · Multi-AZ · ASG self-healing و Read Replica للقراءة · SQS يفصل المكونات و SNS ينشر للجميع.

24%

Performing

## تصميم Architectures عالية الأداء

3

الفكرة: تختار الخدمة الصح للشغلانة الصح (right tool for the job)، وتستخدم الـ caching، والـ scaling، والخدمات المُدارة (managed/serverless) عشان توصل لأعلى أداء بأقل مجهود تشغيلي.

Compute 

## عائلات EC2

T = Burstable

G / P = GPU

I / D = Storage (تخزين)

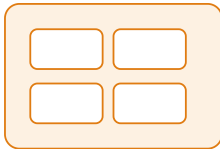
M = General (متوازن)

R = RAM (ذاكرة)

C = Compute (معالجة)

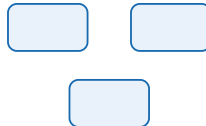
**Placement Groups** بتنحكم في توزيع الـ instances فيزيائيًا:

## Cluster



قربين جدًا - latency أقل

## Spread



كل واحد على rack منفصل - أعلى توافر

## Partition



مجموعات منفصلة - Hadoop/Kafka

Cluster لأقل latency · Spread لأعلى توافر (أحمال حرجة قليلة) · Partition لأنظمة البيانات الموزعة الكبيرة

bandwidth: **Enhanced Networking** (ENA / SR-IOV) أعلى و latency أقل على مستوى الـ NIC.

## Lambda

- الذاكرة = القوة: زود الـ **memory** يزيد الـ CPU معاها (Power Tuning لإيجاد النقطة المثلى).
- حدود مهمة: أقصى مدة تنفيذ **15 دقيقة** · أقصى ذاكرة **10GB**.
- **Provisioned Concurrency**: تشغل instances جاهزة مسبقًا لتجنب الـ cold start.
- **Layers** لمشاركة مكتبات، و **Container Images** لحزم لحد 10GB.

## Containers + Beanstalk

ECS · EKS · Fargate

| إيه هو  |                                                                               |
|---------|-------------------------------------------------------------------------------|
| ECS     | تنسيق containers خاص بـ AWS – launch type إما EC2 (تدير السيرفرات) أو Fargate |
| EKS     | Kubernetes مُدار – لو عايز Kubernetes قياسي / portability                     |
| Fargate | تشغيل containers بدون إدارة سيرفرات (ECS/EKS J serverless)                    |

*Rolling with batch* ·· (دفعات) *Rolling* · (أسرع بس فيه downtime) **Elastic Beanstalk Deployment Policies: All at Once** (آمن، بدون downtime). *Immutable / Blue-Green*

## أداء التخزين

### EBS volumes أنواع

| النوع             | الفئة           | امتى                                                       |
|-------------------|-----------------|------------------------------------------------------------|
| gp3 / gp2         | SSD عام         | <b>الافتراضي</b> لمعظم الأحمال (gp3 تفصل ال IOPS عن السعة) |
| io2 Block Express | SSD عالي الأداء | قواعد بيانات حرجية محتاجة IOPS عالية + متانة عالية         |
| st1               | HDD throughput  | big data / logs (وصول تسلسلي، throughput مثلث IOPS)        |
| sc1               | HDD بارد        | الأرخص – بيانات نادرة الوصول                               |

- **IOPS vs Throughput:** IOPS = عدد العمليات/ثانية (قواعد بيانات/ملفات صغيرة)؛ MB/s = Throughput (ملفات كبيرة/تسلسلي).
- **EBS Multi-Attach:** نفس ال volume على كذا io1/io2 instance – **بس وفي نفس ال AZ**.
- **Instance Store:** تخزين مؤقت (ephemeral) ملاصق فيزيائيًا – IOPS عالية جدًا بس **بيضيع** لو اتعمل stop.
- **EFS Modes:** Performance (General Purpose / Max I/O) و Throughput (Bursting / Provisioned / Elastic).
- **S3 Performance:** parallelization بال multipart upload · prefixes للملفات الكبيرة · **Transfer Acceleration** عبر edge للرفع من بعيد.
- **S3 Select / Glacier Select:** استعلام SQL على جزء من الأوبجكت بدون تنزيله كله.

## أداء قواعد البيانات

### ElastiCache: Redis vs Memcached

| Redis                                                              | Memcached                                      |                |
|--------------------------------------------------------------------|------------------------------------------------|----------------|
| Multi-AZ · replication · persistence · هياكل بيانات غنية · pub/sub | cache · multi-threaded مؤقت بحت بسيط           | <b>الميزات</b> |
| محتاج توافر/استمرارية أو leaderboards/queues                       | cache بسيط قابل للتوسع أفقيًا بدون persistence | <b>امتى</b>    |

| Write-Through                               | Lazy Loading                             |                     |
|---------------------------------------------|------------------------------------------|---------------------|
| عند كل كتابة للـ DB                         | عند القراءة فقط (cache miss → يجب ويخزن) | إمتى يكتب الـ cache |
| الداتا في الـ cache دائماً حديثة            | يخزن المطلوب فعلاً بس                    | الميزة              |
| يكتب بيانات ممكن متقراش (TTL يحل المشكلتين) | أول قراءة بطيئة + احتمال بيانات قديمة    | العيب               |

- **DynamoDB DAX:** cache داخل الذاكرة لـ DynamoDB بـ latency بالـ microseconds.
- **Partition Key:** اختاره موزع كويس عشان تتجنب «hot partition» (ترافيك مكّس على partition واحد).
- **DynamoDB Streams + Lambda:** تشغل دالة عند أي تغيير في الجدول (event-driven).
- **RDS Proxy:** connection pooling – بيقلل ضغط الاتصالات وممتاز مع Lambda.
- **Aurora Parallel Query:** يوزع الاستعلام التحليلي على طبقة التخزين لسرعة أكبر.

قواعد البيانات المتخصصة (Purpose-built) – الكلمة المفتاحية تحدد الإجابة

| الخدمة     | النوع                          | امتى تختارها                                            |
|------------|--------------------------------|---------------------------------------------------------|
| Redshift   | Data Warehouse (columnar, MPP) | تحليلات OLAP على بيانات ضخمة · Spectrum يستعلم على S3   |
| Neptune    | Graph                          | علاقات معقدة · fraud · recommendation · social networks |
| DocumentDB | Document                       | متوافق مع MongoDB                                       |
| Keyspaces  | Wide-column                    | متوافق مع Cassandra                                     |
| Timestream | Time-series                    | بيانات IoT/أجهزة استشعار بالزمن                         |
| QLDB       | Ledger                         | سجل غير قابل للتعديل (immutable) قابل للتحقق التشفيري   |

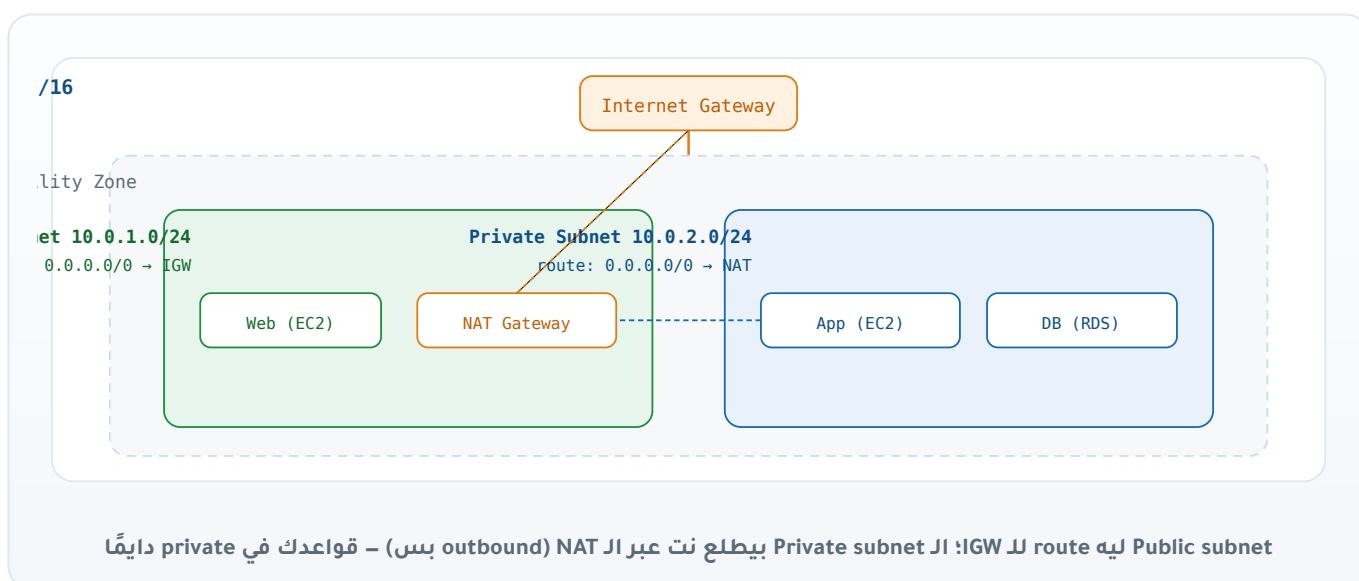
| الخدمة                 | بتعمل إيه                                                             |
|------------------------|-----------------------------------------------------------------------|
| Kinesis Data Streams   | تدقق <b>real-time</b> بـ shards – إنت تتحكم في المعالجة والاحتفاظ     |
| Kinesis Data Firehose  | تحميل <b>near-real-time</b> بدون كود مباشرة لـ S3/Redshift/OpenSearch |
| Kinesis Data Analytics | استعلامات SQL/Flink على الـ streams لحظيًا                            |
| MSK                    | مُدار Apache Kafka                                                    |
| Athena                 | <b>serverless</b> SQL مباشرة على S3 (بتدفع لكل query)                 |
| Glue                   | ETL مُدار + Data Catalog + Crawler                                    |
| EMR                    | Hadoop/Spark/Hive لمعالجة كميات بيانات ضخمة                           |
| OpenSearch             | بحث وتحليل logs (Elasticsearch)                                       |
| Lake Formation         | بناء وإدارة Data Lake وحوكمة الصلاحيات                                |

## نقطة امتحان 💡

«أبسط طريقة أحط streaming data في S3/Redshift بدون كود» → **Firehose**. «استعلام SQL على ملفات في S3 من غير سيرفرات» → «Data Warehouse» → **Athena** للتحليلات → **Redshift**.

## أداء الشبكة 🌐

## تشرح الـ VPC



| المعنى / امتي                                                                   |                            |
|---------------------------------------------------------------------------------|----------------------------|
| ال Gateway مُدار وعالي التوافر (مفصل): ال Instance أرخص بس إنت تديره            | NAT Gateway vs Instance    |
| ربط VPC ين 1:1 – غير متعدّد (non-transitive)، منش بيتوسّع كويس مع كتر الشبكات   | VPC Peering                |
| hub-and-spoke ب transitive routing – يربط مئات ال VPCs وال on-prem في مكان واحد | Transit Gateway            |
| خط فيزيائي مخصّص (Gbps 1/10) – أداء ثابت وأمان للربط مع ال data center          | Direct Connect (DX)        |
| VPN كمسار احتياطي مشفّر فوق/بجانب ال DX                                         | DX + VPN                   |
| Site-to-Site يربط شبكة كاملة (on-prem): Client VPN يربط مستخدمين أفراد          | Site-to-Site vs Client VPN |

### ⚠ خلي بالك

لو السؤال فيه «عشرات/مئات ال VPCs محتاجة تتكلم مع بعض» → الإجابة **Transit Gateway** مش VPC Peering، لأن ال Peering غير متعدّد ومش عملي على نطاق كبير.

## API و Serverless



المعمارية ال serverless الكلاسيكية – بدون أي سيرفرات تديرها

API Gateway + Lambda + DynamoDB: توسّع تلقائي، دفع بالاستخدام، صفر إدارة سيرفرات

### أنواع API Gateway

| النوع         | امتي                                                              |
|---------------|-------------------------------------------------------------------|
| REST API      | مميزات كاملة (API keys, usage plans, caching, request validation) |
| HTTP API      | أبسط وأرخص وأسرع – لمعظم حالات ال Lambda/HTTP البسيطة             |
| WebSocket API | اتصال ثنائي الاتجاه real-time (شات/إشعارات)                       |

- **API Gateway:** caching لتقليل الجمل، usage plansg throttling للتحكم في المعدّل.
- **AppSync:** GraphQL مُدار مع real-time subscriptions – مرونة في جلب البيانات للموبايل/الويب.

### 💡 خلاصة Domain 3 في سطر

اختر العائلة الصح (C/R/I/G) · cache ب gp3 · ElastiCache/DAX افتراضي · DB المتخصص حسب الكلمة المفتاحية · Transit Gateway للشبكات الكثير · serverless = API GW + Lambda + DynamoDB.

20%

Cost

## تصميم Architectures موفرة للتكلفة

4

المبدأ: تدفع على اللي بتستخدمه بس، تختار نموذج التسعير الصح حسب نمط الجمل، وتنقل البيانات للطبقة الأرخص المناسبة. القاعدة: right-size الأول، وبعدين اعمل commitment.

## نماذج تسعير EC2

← أعلى مرونة / أعلى تكلفة أعلى التزام / أقل تكلفة →

## On-Demand

بدون التزام · الأعلى  
أحمال قصيرة/غير متوقعة

## Reserved / Savings

الالتزام 1 أو 3 سنين  
خصم حتى 72% · جمل ثابت

## Spot

خصم حتى 90%  
قابل للمقاطعة (إشعار دقيقتين)

## Dedicated

عزل/ BYOL/ compliance

كل ما زاد التزامك أو تحمّلك للمقاطعة، قلّت التكلفة – اخلط بينهم حسب طبيعة كل جمل

## Reserved vs Savings Plans vs Spot – امتي تستخدم كل واحد

| النموذج                   | الفكرة                                                                        | امتي                                                               |
|---------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------|
| On-Demand                 | بالساعة/الثانية بدون التزام                                                   | تطوير/اختبار · أحمال قصيرة وغير متوقعة                             |
| Reserved Instances        | الالتزام بنوع instance محدد (Standard أرخص · Convertible يسمح بتغيير العائلة) | جمل ثابت معروف لمدة طويلة                                          |
| Savings Plans             | الالتزام بمبلغ \$/ساعة (Compute أعلى مرونة · EC2 Instance أعلى خصم)           | أمرن من RI لنفس الخصم – جمل ثابت<br>بمرونة في التغيير              |
| Spot                      | سعة فائضة بخصم كبير، AWS تقدر تسحبها بإشعار دقيقتين                           | أحمال متحمّلة للانقطاع (batch · big data · CI/CD · fault-tolerant) |
| Dedicated Hosts/Instances | عتاد فيزيائي مخصّص                                                            | compliance يفرض عزل أو ترخيص BYOL<br>مربوط بالعتاد                 |

## نقطة امتحان

«جمل batch ممكن يتقطع ومحتاج أوفر لأقصى درجة» → Spot. «سيرفرات أساسية شغالة 24/7 لمدة سنة+» → Savings.  
Plans / Reserved. «ترخيص Windows/Oracle مربوط بعدد cores فيزيائية» → Dedicated Hosts.

## فئات تخزين S3

**Standard**

فوري • متكرر

**Standard-IA**

فوري • نادر (30 يوم)

**Glacier Instant**

فوري • أرشيف (90 يوم)

**Glacier Flexible**

دقائق-ساعات (90)

**Deep Archive**

48-12 ساعة (180)

Intelligent-Tiering فوق كل ده: ينقل الأوبجكت تلقائيًا بين الطبقات حسب الاستخدام (بدون رسوم استرجاع)

كل ما نزلت لطبقة أبرد: تكلفة التخزين أقل، بس الاسترجاع أبطأ/أعلى وفيه حد أدنى لمدة التخزين

## تفاصيل فئات S3 (الأرقام دي بيتسأل عنها)

| الفئة                      | الاسترجاع   | أقل مدة | امتى                                            |
|----------------------------|-------------|---------|-------------------------------------------------|
| Standard                   | فوري        | –       | بيانات نشطة متكررة الوصول                       |
| Intelligent-Tiering        | فوري        | –       | <b>نمط وصول غير معروف/متغير</b> – يحرك تلقائيًا |
| Standard-IA                | فوري        | 30 يوم  | نادر الوصول بس محتاج فوري + متعدد AZ            |
| One Zone-IA                | فوري        | 30 يوم  | نادر + قابل لإعادة الإنشاء (AZ واحد، أرخص)      |
| Glacier Instant Retrieval  | فوري (ms)   | 90 يوم  | أرشيف بيتقري مرة كل شوية بس بسرعة               |
| Glacier Flexible Retrieval | دقائق-ساعات | 90 يوم  | أرشيف يستحمل تأخير (Expedited/Standard/Bulk)    |
| Glacier Deep Archive       | 48-12 ساعة  | 180 يوم | <b>الأرخص</b> – أرشيف طويل الأمد (امثال قانوني) |

**S3 Lifecycle Policies:** قواعد بتنقل الأوبجكت تلقائيًا لطبقة أبرد بعد مدة (transition) أو بتحذفه (expiration) – أهم أداة لتوفير تكلفة التخزين.

## توفير التكلفة في Serverless

- **Lambda** بتدفع على عدد ال requests × المدة × الذاكرة – صفر تكلفة وقت الخمول.
- **Lambda Power Tuning:** تختبر إعدادات ذاكرة مختلفة لإيجاد توازن السعر/الأداء.
- **Fargate vs EC2:** Fargate أوفر للأحمال المتقطعة (مفیش سيرفر فاضي)، بس EC2 أوفر للأحمال الثابتة العالية المستمرة.
- **Lambda vs EC2 للأحمال المتقطعة (bursty):** Lambda أوفر بكتير لأنك مش بتدفع وقت السكون.

## أدوات إدارة التكلفة

كل أداة بتعمل إيه

| الأداة               | الوظيفة                                                              |
|----------------------|----------------------------------------------------------------------|
| Cost Explorer        | تصوّر وتحلّ وتتنبأ بالتكاليف (رؤية تاريخية ومستقبلية)                |
| AWS Budgets          | تحدّد ميزانية وتأخذ <b>تنبيهات</b> عند تجاوز التكلفة/الاستخدام/الحجز |
| Trusted Advisor      | توصيات عبر 5 فئات: التكلفة + الأمان + الأداء + التوافر + حدود الخدمة |
| Compute Optimizer    | توصيات <b>right-sizing</b> بال ML لـ EC2/EBS/Lambda                  |
| S3 Storage Lens      | تحليلات شاملة لاستخدام وتكلفة التخزين عبر الحساب/المؤسسة             |
| Cost Allocation Tags | tags بتقسّم التكلفة حسب المشروع/القسم في الفواتير والتقارير          |

نقطة امتحان 💡

«عايز تنبيه لو الفاتورة عدّت مبلغ» → **Budgets**. «عايز أحلّ وأتنبأ بالاتجاه» → **Cost Explorer**. «توصيات لتصغير سيرفرات أكبر من اللازم» → **Compute Optimizer**.

🏠 أنماط معمارية لتقليل التكلفة

قواعد التكلفة المعمارية

| الموضوع                     | القاعدة                                                                                                                  |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Data Transfer               | الدخول (in) عادةً مجاني · الخروج للإنترنت (egress) <b>مدفوع</b> · cross-region و cross-AZ مدفوع – قلّل النقل غير الضروري |
| CloudFront لتقليل ال egress | ال caching عند ال edge يقلّل خروج البيانات من S3/الأصل → فاتورة أقل                                                      |
| S3 vs EBS vs EFS            | S3 أرخص للأوبجكتس · EBS لـ instance واحد · EFS مشترك (أعلى) – لا تستخدم block storage لو object كفاية                    |
| RDS vs Aurora vs DynamoDB   | DynamoDB غالباً أوفر للأحمال غير المنتظمة (on-demand capacity): علائقي ثابت → RDS/Aurora                                 |
| Single vs Multi-AZ          | Multi-AZ بيضاعف تكلفة ال DB تقريباً – استخدمه للإنتاج اللي محتاج توافر بس                                                |
| Reserved قبل Right-size     | <b>صغّر السيرفرات الأول</b> ، وبعدين اعمل التزام طويل – عشان متلتزمش بحجم غلط                                            |

✅ خلاصة 4 Domain في سطر

Spot للمتقطّع · Savings Plans للثابت · Intelligent-Tiering لو نمط الوصول مجهول · Lifecycle ينقل للأبرد · CloudFront يقلّل ال right-size قبل أي التزام. egress



Cross-Domain

## Well-Architected + الأنماط المعمارية

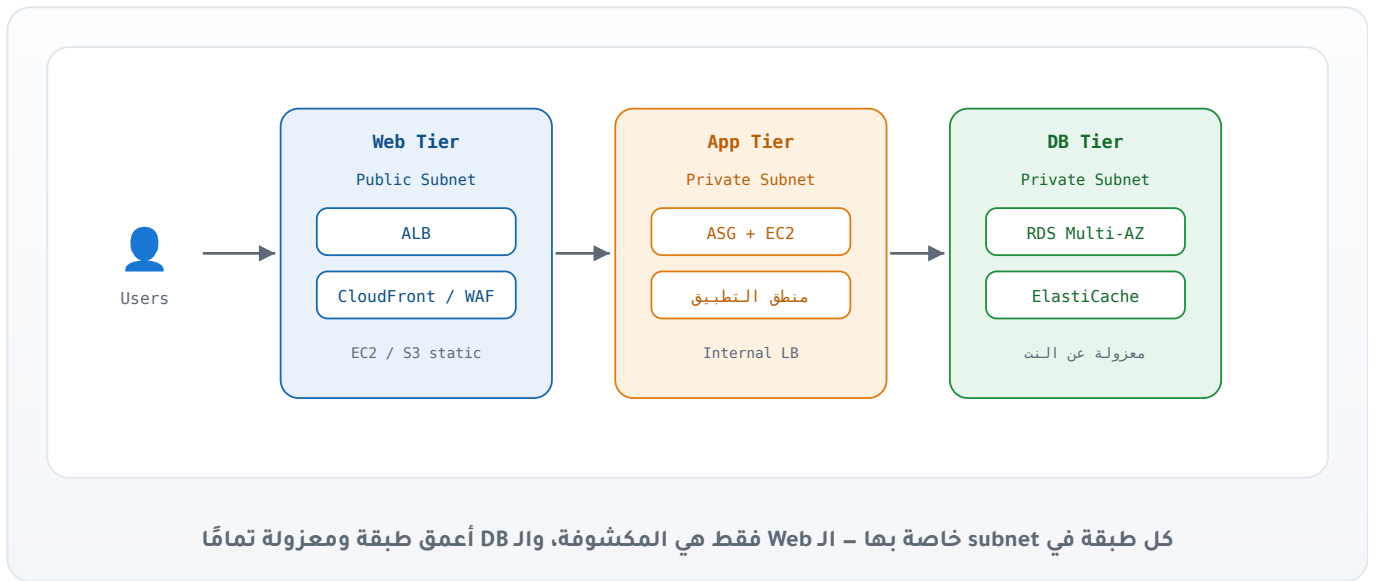
5

الأنماط دي مش domain لوحدها، لكنها بتظهر في أسئلة كتير عبر ال domains الأربعة. الامتحان بيحب السيناريو اللي بيقول «الشركة عايزة كذا» وانت تختار النمط الصح. خليك فاهم **امتى** تستخدم كل نمط مش بس تعريفه.

### Three-Tier Architecture



تقسيم التطبيق لـ 3 طبقات منفصلة: **Web** (الواجهة) و **App** (المنطق) و **DB** (البيانات). كل طبقة بتـ scale وتتأمن لوحدها، والفصل ده بيسهل الصيانة والتوسيع.



### Serverless & Microservices



#### Serverless Architecture

API GW + Lambda + DynamoDB

مفيش سيرفرات تديرها – **API Gateway** بيستقبل الطلب، **Lambda** بتنفذ المنطق، و **DynamoDB** تخزن البيانات. بتدفع على الاستخدام بس، وال scaling تلقائي بالكامل. مثالي للأحمال المتقطعة وغير المتوقعة.

#### Microservices on ECS / EKS

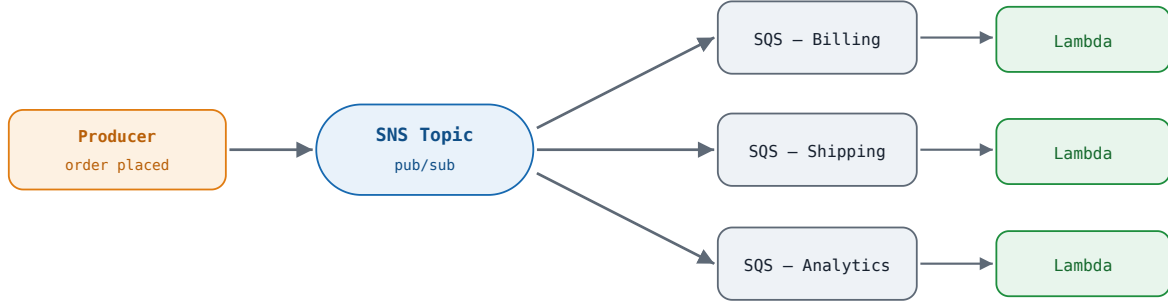
تقسيم التطبيق لخدمات صغيرة مستقلة، كل واحدة بتـ deploy وتـ scale لوحدها. **ECS/Fargate** أبسط في الإدارة، و **EKS** لو محتاج Kubernetes. بتتكلم مع بعضها عبر API أو messaging.

### Event-Driven • Fan-out • CQRS



المكونات بتتواصل عبر **events** بدل ما تنادي بعضها مباشرة (decoupling). ده بيخلي النظام مرن ومتحمل للأعطال، وكل جزء يـ scale لوحده.

(SQS → SNS) Fan-out Pattern



رسالة واحدة من ال Producer توصل لكل ال SQS queues في نفس الوقت – كل خدمة تعالج نسختها باستقلال

### الأنماط القائمة على الأحداث – الفكرة والاستخدام

| النمط        | الفكرة                                                                 | امتى تستخدمه                                                                                                             |
|--------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Event-Driven | المكونات تتواصل عبر events عبر SNS + SQS + Lambda بدون ربط مباشر       | أنظمة تحتاج decoupling ومرونة عالية وتحمل أعطال                                                                          |
| Fan-out      | حدث واحد ينسخ لعدة مستهلكين: SNS topic → عدة SQS queues                | نفس الحدث محتاج أكثر من معالجة مستقلة (فوترة + شحن + تحليلات)                                                            |
| CQRS         | فصل عمليات الكتابة عن القراءة (Command vs Query) في مودلز/مخازن مختلفة | قراءات أضخم بكثير من الكتابات، أو احتياج تحسين كل جانب لوحده (مثلًا DynamoDB للكتابة + read replicas/OpenSearch للقراءة) |

#### نقطة امتحان 💡

«حدث واحد لازم يوصل لأكثر من نظام مستقل» → (SQS → SNS) Fan-out. «عايز decoupling بحيث لو مستهلك وقع الرسائل متضيعش» → SQS بين المنتج والمستهلك.

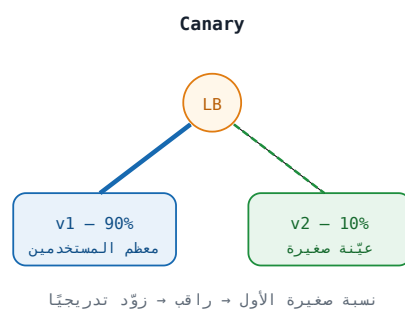
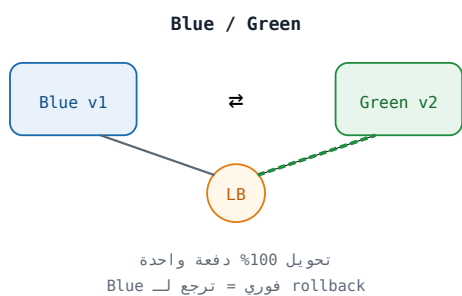
### Strangler Fig – نمط هجرة الأنظمة القديمة 🌿

بدل ما تعيد كتابة النظام القديم (monolith) دفعة واحدة – وده خطر – بتبني الجديد **حواليه تدريجيًا**: تحوّل feature ورا feature للنظام الجديد، وال legacy يفضل شغال لحد ما يت «خنق» بالكامل ويتشال. الاسم جاي من شجرة التين الخانقة. على AWS غالبًا بتحت **API Gateway / ALB** قدام، يوجّه كل route للنظام الصح (قديم أو جديد) أثناء الانتقال.

#### ✓ له مهم

هجرة آمنة ومتدرّجة بأقل مخاطرة وبدون «big bang rewrite» – تقدر ترجع بسهولة لو حاجة وقعت.

### أنماط ال Deployment – Blue/Green vs Canary 🚀



Blue/Green = بيئتان كاملتان وتبديل لحظي · Canary = توجيه نسبة صغيرة للنسخة الجديدة ثم التوسع تدريجيًا

## أنماط ال Deployment – مقارنة

| النمط       | إزاي يشتغل                                 | المميزات                        | امتي                                 |
|-------------|--------------------------------------------|---------------------------------|--------------------------------------|
| All at Once | تحديث كل السيرفرات مرة واحدة               | أسرع وأرخص                      | downtime مقبول · بيئات تطوير         |
| Rolling     | تحديث دفعات صغيرة بالتتابع                 | بدون بيئة إضافية                | تقليل التأثير مع توفير الموارد       |
| Blue/Green  | بيئتان كاملتان، تبديل ال 100% traffic      | rollback فوري · صفر downtime    | إنتاج حساس يحتاج أمان وتراجع سريع    |
| Canary      | نسبة صغيرة للنسخة الجديدة ثم زيادة تدريجية | اكتشاف مشاكل مبكرًا بأقل مخاطرة | عايز تختبر على مستخدمين حقيقيين بحذر |

## Pillars 6 JI – Well-Architected Framework 🏛️

إطار AWS لتقييم وتحسين أي architecture. الامتحان بيربط أسئلة كثير بالأركان دي – لو فهمت كل ركن بيركز على إيه. هتختار الحل الصح بسهولة.

الأركان الستة للـ Well-Architected Framework

| الركن                  | يركّز على                                                    | أمثلة خدمات/ممارسات                                     |
|------------------------|--------------------------------------------------------------|---------------------------------------------------------|
| Operational Excellence | تشغيل ومراقبة الأنظمة وتحسين العمليات باستمرار · الأتمتة     | laC · CloudWatch · CloudFormation runbooks              |
| Security               | حماية البيانات والأنظمة · least privilege · التشفير · التتبع | GuardDuty · KMS · IAM · CloudTrail                      |
| Reliability            | التعافي من الأعطال · تلبية الطلب · Multi-AZ وال back-ups     | Multi-AZ · Auto Scaling · Route 53 health checks        |
| Performance Efficiency | استخدام الموارد بكفاءة واختيار النوع الصح لكل جمل            | caching · serverless · DBs · instance الصبح · ال مخصصة  |
| Cost Optimization      | تجنّب الإنفاق غير الضروري · right-sizing · نماذج التسعير     | Spot · Savings Plans · Cost Explorer · lifecycle        |
| Sustainability         | تقليل الأثر البيئي للأحمال · استغلال أمثل للموارد            | regions كفاءة · serverless · right-sizing · تقليل الهدر |

✓ خلاصة الأنماط في سطر

Three-tier للفصل · Serverless للأحمال المتقطعة · Event-driven/Fan-out للـ Strangler Fig · decoupling للهجرة الآمنة · Blue/Green للتراجع الفوري · Canary للاختبار التدريجي · وال 6 pillars بوصلتك في أي قرار تصميم.

خطة متدرجة تبدأ بالأساسيات (IAM, VPC, EC2) وتنتهي بال mock exams ومراجعة نقاط الضعف. الالتزام بالترتيب ده بيبي المفاهيم فوق بعضها صح. عدّلها على وقتك المتاح، بس حافظ على الأسبوع الأخير للامتحانات التجريبية.

## الجدول الأسبوعي المقترح

### خطة ال 8 أسابيع

| الأسبوع   | التركيز                              | ملاحظة                                   |
|-----------|--------------------------------------|------------------------------------------|
| الأسبوع 1 | IAM · VPC · EC2                      | الأساس – افهمه كويس قبل ما تكمل          |
| الأسبوع 2 | S3 · RDS · Aurora · DynamoDB         | التخزين وقواعد البيانات                  |
| الأسبوع 3 | ELB · ASG · Route 53 · CloudFront    | التوزيع والتوسّع وال DNS                 |
| الأسبوع 4 | Security Services · Encryption       | Domain 1 بالكامل – أكبر وزن (30%)        |
| الأسبوع 5 | Serverless · Containers · Analytics  | Lambda · ECS/EKS · Kinesis · Athena      |
| الأسبوع 6 | Networking Advanced · DR Strategies  | Transit Gateway · Direct Connect · DR    |
| الأسبوع 7 | Cost Optimization · Well-Architected | pillars 6 ال + Domain 4                  |
| الأسبوع 8 | Mock Exams + مراجعة نقاط الضعف       | الأهم – اتدرب على أسئلة كثير وحلّ غلطاتك |

## أفضل المصادر

### المصادر الموصى بها لـ SAA-C03

| المصدر                     | النوع            | ليه                                                              |
|----------------------------|------------------|------------------------------------------------------------------|
| Stephane Maarek            | كورس Udemy       | أشهر كورس للشهادة – شرح عملي شامل لكل الخدمات                    |
| Tutorials Dojo (Jon Bonso) | امتحانات تجريبية | أقرب أسئلة لأسلوب الامتحان الحقيقي – شروحات ممتازة لكل إجابة     |
| AWS Skill Builder          | أسئلة رسمية      | أسئلة تدريبية رسمية من AWS نفسها                                 |
| AWS WhitePapers            | مستندات          | خصوصًا Well-Architected Framework و Security Best Practices      |
| AWS FAQs                   | أسئلة شائعة      | اقرأ ال FAQ لكل service مهمة – كثير من تفاصيل الأسئلة بييجي منها |

### 💡 نصيحة المذاكرة الذهبية

الكورس بيديك الأساس، لكن **الامتحانات التجريبية هي التي بتنجحك**. ذاكر، ثم حلّ practice exams، وكل سؤال تغلط فيه ارجع للمفهوم وافهمه ليه. كرر لحد ما توصل ثبات فوق 80% في التجريبي.

### 🚀 بالتوفيق في الامتحان يا أمار

افتكر القاعدة: في القاعدة اقرأ السؤال كويس، حدّد **الكلمة المفتاحية** (lowest ؟highly available ؟most cost-effective ؟least operational overhead ؟latency) – الكلمة دي بتوجهك للإجابة الصح.

مبني على ال Study Checklist الخاص بك · الإصدار الحالي للامتحان: SAA-C03